FilsPay pitch deck

Draft for `AlgoDuo_FilsPay_Slides.pdf`. 16:9. Eight content slides, a cover, a closing slide, and appendix slides. Timing target: 4 minutes, so about 45 to 60 seconds per content slide. One message and one evidence number per slide. Everything here matches the submission document; re-verify at freeze.

Cover

Algo Duo

FilsPay: money that adds up

TODO-PLACEHOLDER: member names

Live: TODO-PLACEHOLDER: checkout, dashboard, admin, API Fork: TODO-PLACEHOLDER: public fork URL Commit: TODO-PLACEHOLDER: short SHA

Slide 1: Team

Who did what

Member	Owned
TODO-PLACEHOLDER	TODO-PLACEHOLDER: settlement Go lane
TODO-PLACEHOLDER	TODO-PLACEHOLDER: API core TypeScript lane
TODO-PLACEHOLDER	TODO-PLACEHOLDER: web, admin, docs, submission

TODO-PLACEHOLDER: one sentence on how the team split the work.

Slide 2: Root cause

The money contract was broken in three

The money contract was broken in three places

`.cursorrules` and `CLAUDE.md` say cents. The money package and its tests say integer fils. The Go payout path uses `float32`.

> Same defect, three fingerprints: a 10x CSV import, payouts that drift by a fils, and refunds that never reach settlement.

Evidence: `parseBhdToFils("4.75") === 4750` (`packages/money/src/fils.test.ts:172`); `payable.go:11` uses `float32`.

Symptoms this explains: split-capture totals, reconciliation gaps, wrong refundable balances, and the 10x import.

Slide 3: Key findings

Four defects, ranked by impact

CRITICAL: the first two are money-path defects. Fix them or say plainly that they are open.

#	Finding	One line of evidence
1	Payouts drift above 2^24 fils	`payable.go:11` computes through `float32`; 16,777,200 fils returns 16,777,199 [v]
2	Refunds never settle	`routes/refunds.ts:24` returns 501; no refund claim step in CloseBatch [v]
3	Payment links leak key metadata and accept expired payment	link payload exposes key prefix and hash; no server-side expiry gate [lane: 2026-10-30]
4	Webhooks can duplicate across replicas	`LISTEN` race; duplicate payout notifications in support tickets [lane: 2026-10-30]

Slide 4: Fixes and proof

Eight fixes, each with a test or a build

Fix	Proof
Reporting export now requires merchant auth	`reportingAuth.test.ts`: 401 anonymous, 200 authenticated
Fresh database migrates cleanly	66 files, 0 errors on `filspay_fresh`
Dashboard no longer crashes on negatives	`bhdSigned()`; web typecheck and build green
`refundedFils` reaches the client	serializer plus web type; build green
Bulk approve cannot touch rejected merchants	new `bulkApproveGuard.test.ts`
Admin batch date uses Bahrain local time	typecheck and build green
Settlement statuses readable and focusable	chips, responsive rules, focus ring

Baseline held: `bun test` 418 pass, 16 fail, all pre-existing; `go test ./...` green.

TODO-PLACEHOLDER: before and after screenshots, one pair on this slide.

Slide 5: Live demo

Full and partial refund, end to end

Full and partial refund, end to end

1. Open the demo payment link and pay at `/pay/:token`.
2. Sign in as the merchant with OTP.
3. Open the transaction and the refund drawer.
4. Issue a full refund, then a partial refund on another transaction.
5. Show the correct amounts in the dashboard and in the settlement report.

Cross-checks: `settlement export-scheme-file --batch=<uuid>`, `settlement rebuild-balances`, `bun apps/reporting/src/index.ts catch-up`.

Fail-safe: TODO-PLACEHOLDER: screenshots of every step, captured at rehearsal.

> The refund route is 501 in this tree. Run the demo only from the fork build that carries the refund lane, and say that plainly if it is not ready. [v: `routes/refunds.ts:24`]

Slide 6: Load test

34 requests per second measured, 30 to 60 estimated

```
browsers → host proxy → api :3000 → postgres
                                  ⟼ settlement :8091 (internal)
                                  ⟼ reporting (outbox consumer)
```

Metric	Measured
Throughput	20,471 requests in 600 s, about 34.1 req/s, 28 VUs
list_transactions	p50 41 ms, p95 112 ms, 0 errors
settlement_summary	p50 88 ms, p95 240 ms, 0 errors
export_csv	p50 2.21 s, 0 errors
Peak RSS	236 MB of 512 MB, 0 OOM kills

Bottleneck: Postgres pool of 5 per API, then export memory, then the missing `transaction_events` index. Recovery: pool fail-fast plus idempotent batches.

Slide 7: How you used AI

The config said cents. The tests said fils

The coding said cents. The tests said fils.

Setup: `AGENTS.md`, `CLAUDE.md`, `.cursorrules`, an `.omo/` planning workspace, and `scripts/`.

The AI was wrong: `.cursorrules:5-7` and `CLAUDE.md:22-24` claim integer cents, 1 BD = 100 cents.

How we caught it: the money package says integer fils (`packages/money/src/fils.ts:1-9`), and `parseBhdToFils("4.75")` returns 4750, not 475.

What changed: TODO-PLACEHOLDER: corrected the convention files and added a test guard, or describe the actual follow-up.

Money right to the fils

TODO-PLACEHOLDER: one sentence on what the team is proudest of.

Status: NEEDS-REAL-TRANSCRIPT. Replace the paraphrase with a real excerpt before presenting.

Fork: TODO-PLACEHOLDER Live: TODO-PLACEHOLDER

Appendix A: Architecture and open risks

What we did not touch, and why

Decision	Why	What is still open
en `packages/money` and its mirror	Finance-approved rounding and test vectors	Any change needs re-certification
v1 501 stubs	Reserved partner features	Refunds stay unavailable until the lane lands
webhook	Ships in v1.9; the queue stalls without an	Add envelope first
		BIGINT migration needs DBA review
windows		Streaming needs resumable downloads
	Each needs a locked test and a money-path review	Payout drift, link expiry, webhook duplication

```
INTERNET
   |
   v
host reverse proxy (Caddy per compose comment)   [claim]
   |
   v  127.0.0.1:3000
+-----------------------------------------+
| api (Fastify)   /v1/*  /healthz  /readyz |
+----------------+------------------+-----+
                 |                  |
                 v                  v
      postgres 16 (5432)     settlement (8091 internal)
                 ^
                 |
      reporting (outbox consumer, no port)
```

Must fix before submission: rotate the committed sandbox acquirer secret in `ops/.env.docker`.

Redeploy: `docker compose -f ops/docker-compose.yml up -d postgres`, then `run --rm migrate`, then `run --rm seed`, then `up -d api settlement reporting`.

Closing

Appendix B: Load-test detail

Scenario	VUs	p50	p95	Max	Errors
export_csv	3	2.21 s	4.18 s	5.93 s	0
list_transactions	20	41 ms	112 ms	488 ms	0
settlement_summary	5	88 ms	240 ms	701 ms	0

Source: `docs/perf/fp-209-loadtest-2026-09.md`, k6 0.52, 10 minutes, 18,432 transactions, API at 0.5 vCPU and 512 MB. TODO-PLACEHOLDER: attach the submission-day rerun.

Appendix C: Test output

TODO-PLACEHOLDER: paste the submission-day tails here.

- `bun test`: 418 pass, 16 fail at baseline, all named and pre-existing.
- `cd apps/settlement && go test ./...`: green.
- Web and admin typecheck and build: green.

Appendix D: Finding detail

TODO-PLACEHOLDER: paste the expanded F1 to F12 table from the submission document, one row per finding with symptom, reproduction, root cause, fix, verification, and effect on data.

Appendix E: Screenshots

TODO-PLACEHOLDER: the full before and after set.

Required pairs: dashboard crash and fix, refundable balance, bulk approve, batch date, settlement status chips, keyboard focus. Plus one anonymous checkout frame and one settlement report frame from the demo.

Appendix F: Verification tags

- `[v: path:line]` read or run in this working tree on 2026-10-30.
- `[lane: 2026-10-30]` verified by the audit lane, re-check at freeze.
- `[plan]` claimed by a plan document, not re-verified.
- `[claim]` unverified, or contradicted by code.
- `TODO-PLACEHOLDER` team fact for the lead.
- `NEEDS-REAL-TRANSCRIPT` replace with a real AI session excerpt.